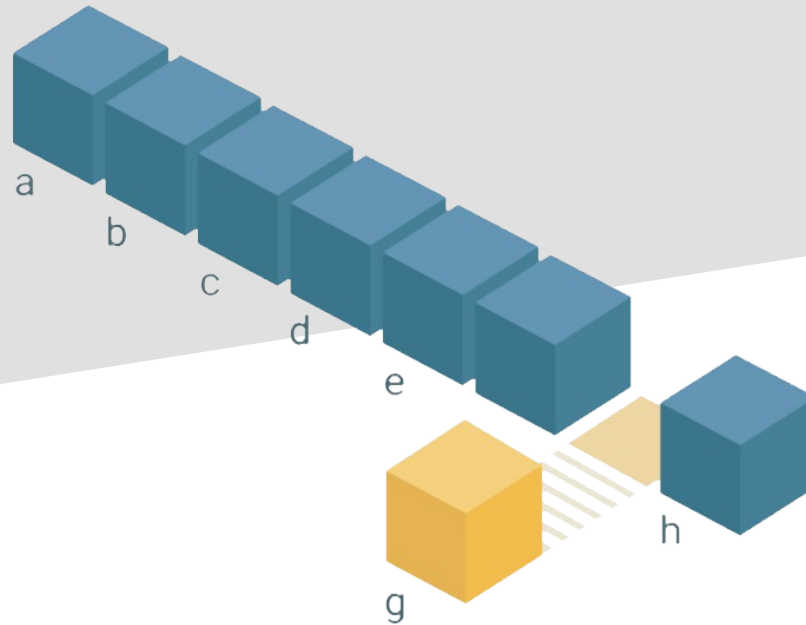




Tema 04. Programación orientada a objetos

Parte 3. Arrays estáticos





Índice

1. Introducción.
2. Definición de arrays.
3. Declaración, creación e inicialización de arrays.
4. Acceso y recorrido de arrays.
5. Arrays como parámetros.
6. Búsqueda de elementos en arrays.
7. Ordenación de elementos.
8. Otros métodos de arrays.



1.- Introducción.

Surgen ante la necesidad de agrupar los datos de un programa en una **estructura de datos**.

Ejemplo:

*Queremos almacenar las notas de un examen de los 30 alumnos de la clase.
¿ 30 variables nota1, nota2.... nota30?*

NO!

Un array o vector de notas donde se almacenen de forma organizada los 30 valores.

índices	0	1	2	3	4	...	30
arrayNotas	6	4	8	9	6	...	3



2.- Definición de arrays.

Un array (vector) es una colección ordenada de elementos del mismo tipo, donde cada elemento está asociado a un índice o posición que ocupa.

Los elementos de un array se almacena en posiciones contiguas de memoria

Un vector queda determinado por:

- Tipo de sus elementos.
- Número de elementos.

Para utilizar un array o vector hay que realizar tres operaciones:





3.- Declaración, creación e inicialización de arrays.

DECLARACIÓN

Como otras variables, antes de poder utilizar un vector, primero se debe declarar.

Se pueden declarar de dos formas diferentes:

- `tipo[] nombre;`
- `tipo nombre[];`

Donde `tipo`, indica el tipo de los elementos del vector, que pueden ser de cualquier tipo (`int`, `double`, `String`, un objeto); y `nombre` es un identificador que nombra al array o vector.

De esta forma `nombre` es una referencia a un vector, aunque el vector todavía no está creado.



3.- Declaración, creación e inicialización de arrays.

DECLARACIÓN

Ejemplos:

```
double[] vectorNotas; // vector de elementos de tipo double
String[] clase; // vector con los nombres de los alumnos
Cuenta[] vectorCuentas; // vector de objetos cuenta
```

También se pueden declarar así:

```
double vectorNotas[];
String[] clase;
Cuenta vectorCuentas[];
```



3.- Declaración, creación e inicialización de arrays.

CREACIÓN

Tras declarar el vector, el siguiente paso es crearlo o construirlo.

Hay que indicar el número de elementos del vector para que se pueda reservar la cantidad de memoria necesaria para contener todos sus elementos

La sintaxis es la siguiente:

- `nombre = new tipo[numeroDeElementos];`

Donde `nombre` es el nombre del vector (previamente declarado); `tipo`, es el tipo de los elementos del vector; y `numeroDeElementos` especifica el número de elementos que tendrá.



3.- Declaración, creación e inicialización de arrays.

CREACIÓN

Ejemplos:

Crear un vector identificado por notas, con 30 elementos de tipo double.

```
double[] vectorNotas;  
vectorNotas = new double[30];
```

Crear un vector de 10 objetos Cuenta.

```
Cuenta vectorDeCuentas[];  
vectorDeCuentas = new Cuenta[10];
```

IMPORTANTE

Cuando se crea un vector de objetos, se ha hecho el new del vector, **no de los objetos**. Se ha reservado memoria para poder almacenar 10 objetos de tipo Cuenta pero todos están ahora a null.



3.- Declaración, creación e inicialización de arrays.

CREACIÓN

Es muy común declarar y crear el vector en una sola línea de código, de la siguiente forma:

- `tipo[] nombre = new tipo[tamanno];`
- `tipo nombre[] = new tipo[tamanno];`

Ejemplo:

Si se define un vector de tamaño 5, con las notas de los alumnos, tendríamos:

```
double[] vectorNotas = new double[30];  
Cuenta vectorCuentas[] = new Cuenta[10];
```



3.- Declaración, creación e inicialización de arrays.

INICIALIZACIÓN

Si no se especifica ningún valor, los elementos de un vector se inicializan automáticamente a unos valores predeterminados (variables numéricas a 0, objetos a null y booleanas a false).

Es posible inicializarlo con los valores que se deseen a la vez que se declaran. En este caso no hace falta hacer el new y el tamaño del vector se entiende por el número de elementos.

Ejemplos:

```
// se crearía el vector con tamaño 8
double[] vectorNotas = {5.5, 6, 5, 8, 3, 9, 10, 8.5 };

// tamaño 5
String[] diasLaborables= {"Lunes", "Martes", "Miercoles",
                           "Jueves", "Viernes"};
```



4.- Acceso y recorrido de arrays.

ACCESO

Para acceder al valor de un elemento de un vector, se utiliza el nombre del vector, seguido de un subíndice entre corchetes.

Los índices van desde la posición 0, al tamaño del vector-1.

Para el ejemplo de las notas:

índices	0	1	2	3	4	5	6	7
vectorNotas	5.5	6	5	8	3	9	10	8.5



4.- Acceso y recorrido de arrays.

ACCESO

Para acceder a la nota del alumno 4 -> `vectorNotas[3]`

Un elemento de un vector no es más que una variable, por tanto, se puede utilizar exactamente igual que cualquier otra variable y operar con ellos. Por ejemplo, en las operaciones que se muestran a continuación intervienen elementos de un vector:

```
double[] vectorNotas = new double[30];  
vectorNotas[5] = 8;  
vectorNotas[9] = vectorNotas[5] + 1;
```



4.- Acceso y recorrido de arrays.

ACCESO

Si se intenta acceder a un elemento del vector, con un subíndice fuera de rango, Java lanzará una excepción del tipo:

ArrayIndexOutOfBoundsException

¿Cómo podemos asegurarnos de no exceder el final de un vector?

Verificando la longitud del mismo, mediante el atributo `length`.

Ejemplo:

```
//devuelve el nº de elementos del vector  
int numeroDeElementos = nombreVector.length;
```



4.- Acceso y recorrido de arrays.

RECORRIDO

Siempre que se necesite realizar un tratamiento sobre todo los elementos de un vector habrá que realizar un **recorrido** del vector.

El programador debe preocuparse de que el índice no se pase de los límites del vector.

Ejemplo:

```
for (int i = 0; i < nombreVector.length; i++) {  
    // Realizar el tratamiento que se desee con  
    // el elemento vector[i]  
}
```



4.- Acceso y recorrido de arrays.

RECORRIDO

Por lo general, cuando se recorre un Array, se recorre de manera completa, “visitando” todos sus elementos. Para simplificar esta codificación podemos usar la estructura **for each**.

Ejemplo:

```
for (String nombre : vectorNombres) {  
    System.out.print(nombre);  
}
```

En cada iteración del bucle, la variable nombre cambiará de valor hasta completar todos los elementos de vectorNombres. Este recorrido se realiza de manera ordenada, desde el primer elemento (índice 0) hasta el último (índice vectorNombres.length).



4.- Acceso y recorrido de arrays.

RECORRIDO FOREACH

Foreach no modifica en ningún caso los elementos originales del array, los recorre y los guarda en una variable del ámbito del for que no está relacionada con los elementos del array. Funciona como un paso de parámetros por valor.



SPOILER ALERT

DESDE UN FOREACH NO PODEMOS MODIFICAR LOS VALORES DEL ARRAY



5.- Arrays como parámetro.

Un vector puede pasarse como parámetro a un método. El paso siempre es por **referencia**, es decir cualquier cambio que se haga en un método sobre los elementos de un vector se conservarán al abandonar dicho método.

- Al pasar un vector como parámetro solo se debe indicar el nombre del vector.
- En el prototipo del método si se indica que es un vector con los corchetes [].

Cambia el
valor original

Llamada por
referencia

VS

NO cambia el
valor original

Llamada por
valor



5.- Arrays como parámetro.

Ejemplo:

```
public static void main(String[] args) {
    double[] vectorNotas = new double[TOTAL_ALUMNOS];
    insertarNotasEnVector(vectorNotas);
    mostrarVectorNotas(vectorNotas);
}

private static void insertarNotasEnVector(double[] vectorNotas) {
    for (int i = 0; i < vectorNotas.length; i++) {
        System.out.println("Introduce la nota del alumno " + (i + 1));
        vectorNotas[i] = Double.parseDouble(teclado.nextLine());
    }
}

private static void mostrarVectorNotas(double[] vectorNotas) {
    for (int i = 0; i < vectorNotas.length; i++) {
        System.out.println("Nota del alumno " + (i + 1) + ":" + vectorNotas[i]);
    }
}
```



6.- Búsqueda de elementos en arrays.

Ejemplo:

```
/*
 * Método para buscar en un vector
 * @param vector
 * @param elementoBuscado
 * @return posición en la que se encuentra el elemento dentro del vector
 *         o -1 si no encuentra el elemento en el vector.
 */
public static int buscarElementoEnVector(double[] vector, double elementoBuscado) {
    int posicion = -1;
    int i = 0;
    while (i < vector.length && posicion == -1) {
        if (vector[i] == elementoBuscado)
            posicion = i;
        else
            i++;
    }
    return posicion;
}
```



7.- Ordenación de elementos.

Arrays.sort

La ordenación de un vector es un tema que se ha estudiado mucho. Hay muchos algoritmos, entre ellos el algoritmo de selección, la burbuja, y el algoritmo del quicksort que es el que implementa Java en su método `Arrays.sort`.

`Arrays.sort` ordena por defecto de forma ascendente. Si quisieras ordenar con otro criterio también existe la forma que ya veremos cuando estudiemos las colecciones.





7.- Ordenación de elementos.

ORDENACIÓN TIPOS BÁSICOS Y STRING

```
public static void main(String[] args) {  
    int[] vectorEnteros = { 5, 22, 30, 5, 6, 8 };  
    String[] vectorNombres = { "LUCIA", "ANA MARIA", "LAURA", "ROSA", "JUAN" };  
  
    System.out.println(Arrays.toString(vectorEnteros));  
    System.out.println(Arrays.toString(vectorNombres));  
  
    Arrays.sort(vectorEnteros);  
    Arrays.sort(vectorNombres);  
  
    System.out.println(Arrays.toString(vectorEnteros));  
    System.out.println(Arrays.toString(vectorNombres));  
}
```

Antes:

[5, 22, 30, 5, 6, 8]

[LUCIA, ANA MARIA, LAURA, ROSA, JUAN]

Después:

[5, 5, 6, 8, 22, 30]

[ANA MARIA, JUAN, LAURA, LUCIA, ROSA]



7.- Ordenación de elementos.

ORDENACIÓN DE UN ARRAY DE OBJETOS

Si necesitamos ordenar un array de objetos (por ejemplo Cuenta) la clase tiene que implementar la interfaz **Comparable**.

```
public class Cuenta implements Comparable<Cuenta>{  
    @Override  
    public int compareTo(Cuenta o) {  
        // TODO Establecer el criterio de ordenación  
        return 0;  
    }  
}
```

Esto significa que debe tener implementado el método **compareTo** que describe el criterio de ordenación (en nuestro ejemplo cuando una cuenta es menor o mayor que otra).



7.- Ordenación de elementos.

MÉTODO compareTo

El método compareTo debe tener el prototipo:

```
public int compareTo ( Objeto otro )
```

Funcionará de esta forma:

- **Devuelve cero** si el objeto this es igual que el objeto otro
- **Devuelve un valor menor que cero** si el objeto this es menor que el objeto otro (para la ordenación)
- **Devuelve un valor mayor que cero** si el objeto this es mayor que el objeto otro (para la ordenación)

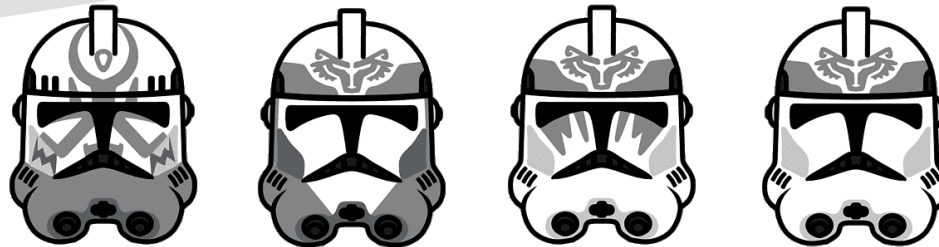


8.- Otros métodos de arrays.

Java maneja los vectores como si fueran objetos, por lo tanto existen una serie de métodos heredados de la clase `Object` que están en el paquete `java.lang`.

Entre ellos cabe destacar:

- **`equals`**. Devuelve `true` si los dos arrays comparados son el mismo, es decir compara las referencias
- **`clone`**. Devuelve un array nuevo con los mismos datos que el array del que se clona.





8.- Otros métodos de arrays.

Ejemplo: (clone e equals)

```
double[] notas1 = { 5.5, 6, 5, 8, 3 };
double[] notas2 = { 5.5, 6, 5, 8, 3 };
double[] notas1Clonado;
double[] notas3;
notas1Clonado = notas1.clone();
notas3 = notas1;
if (notas1.equals(notas2))
    System.out.println("Equals entre notas1 y notas2 dice IGUALES");
else
    System.out.println("Equals entre notas1 y notas2 dice DISTINTOS");
if (notas1.equals(notas1Clonado))
    System.out.println("Equals entre notas1 y notas1Clonado dice IGUALES");
else
    System.out.println("Equals entre notas1 y notas1Clonado dice DISTINTOS");
if (notas1.equals(notas3))
    System.out.println("Equals entre notas1 y notas3 dice IGUALES");
else
    System.out.println("Equals entre notas1 y notas3 dice DISTINTOS");
```



8.- Otros métodos de arrays.

La clase Arrays tiene métodos estáticos que podemos usar pasándoles como parámetros, principalmente, objetos de tipo arrays.

Ejemplo: `Arrays.toString(vectorNombres)` devolverá una cadena con los valores del array, no la referencia. NO OLVIDAR USAR LA CADENA DEVUELTA

```
System.out.println(vectorNombres);  
System.out.println(Arrays.toString(vectorNombres));
```



Console x

```
<terminated> NombreClase [Java Application] C:\Users\y0rg\workspace\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.  
[Ljava.lang.String;@b684286  
[LUCIA, ANA MARIA, LAURA, ROSA, JUAN]
```



Tema 04. Arrays estáticos

